

Premise Selection for Higher-Order ATP Problems



Renaissance
Philanthropy



DEEPER

Keneni W. Tesema, Jan Jakubuv, Martin Suda

Czech Technical University in Prague

Deeper4AI Project

AITP 2026, Aussois , France

Sep 1st, 2026

Intro: Premise selection

- Premise selection is the task of identifying a small subset axioms necessary to prove a given conjecture from a library of thousands of axioms.



- ATP without premise selection



- ATP with premise selection



- Magic

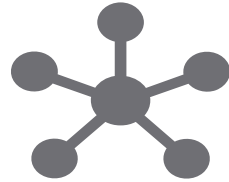
Premise Selection in Higer Order Logic

FOL	HOL
Variables range over individuals	Variables can range over FUNCTIONS(properties etc) : $\forall P. P(\text{agatha}) \rightarrow P(\text{butler})$ + type information
Predicates apply to individuals: <code>hates (agatha, X)</code>	Lambda abstraction : $\lambda x. x + x$ (functions as data)
Functions return individuals: <code>father_of (agatha)</code>	Partial application : <code>hates(agatha)</code> returns a function $\lambda Y. \text{hates}(\text{agatha}, Y)$ @
Quantifiers only over individuals: $\forall X. \text{hates}(\text{agatha}, X)$	Higher-order unification : Matching <code>P(agatha)</code> against <code>hates(agatha, butler)</code> requires finding $P = \lambda X. \text{hates}(X, \text{butler})$
Simple Abstract Syntax Trees(AST) Eg: <code>Ax: hates(agatha, X)</code> <pre>graph TD hates --> agatha hates --> X</pre>	Richer AST Eg: <code>^[X:\$i]: (@(@ (hates, agatha), X))</code> <pre>graph TD lambda["λ"] --> X1["X"] X1 --> at1["@"] at1 --> at2["@"] at1 --> X2["X"] at2 --> agatha["agatha"] at2 --> hates["hates"]</pre>

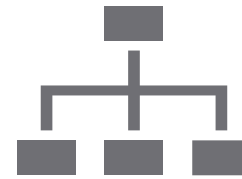
Related works

Technique	Approach	FOL	HOL	Type	Key Limitation
MePo (Meng & Paulson, 2009)	Symbol overlap heuristics	✓	✗	Heuristic	Shallow, no structure
MaSh (Kühlwein et al., 2013)	Naive Bayes + kNN ensemble	✓	✗	Heuristic	Hand-crafted features only
SInE (Hoder & Voronkov, 2011)	Least common symbol trigger (used in Vampire/E-prover)	✓	✗	Heuristic	Approximate, no types
Wang et al. (2017)	Graph embeddings with shared symbol nodes	✓	✓	GNN	First graph-based for HOL
Olšák et al. (2019)	Graph embeddings + property-invariant hypergraph	✓	✗	GNN	Invariant to symbol renaming; focuses on unseen Skolem symbols
ARCG-NN (Liu et al., 2021)	Cross-graph attention (conjecture $\leftrightarrow$ axioms)	✓	✓	GNN	Better relevance encoding
Bansal et al. (2019)	GNN + sequence models (HOL Light)	✓	✓	Hybrid	HOL4/HOL Light specific
Magnushammer (Mikuła et al., 2024)	Contrastive transformer + InfoNCE (SELECT+RERANK)	✓	✓	Sequence	No explicit type structure, (types as atoms, not composed)
Paliwal et al. (2020)	Graph w/ types + binding	✓	✓	GNN	Explicit HOL structure
QUILL (Kogkalidis et al., 2024)	Structured attention + de Bruijn indices	✓	✓	GNN	Dependent types (Agda), not THF

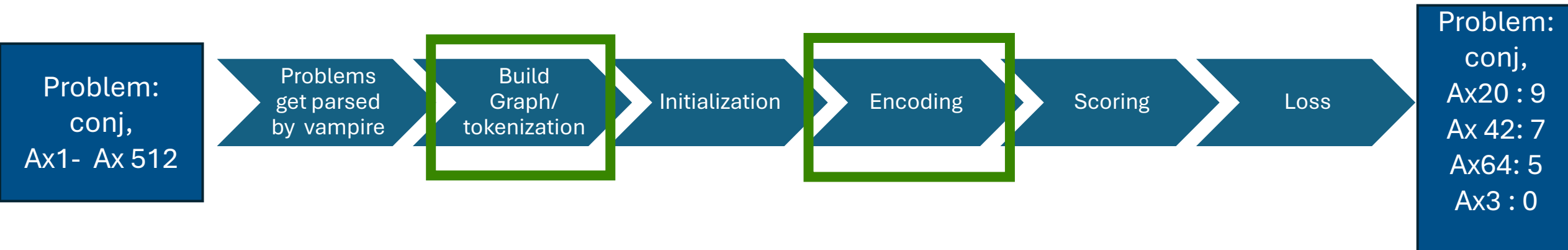
OVERVIEW



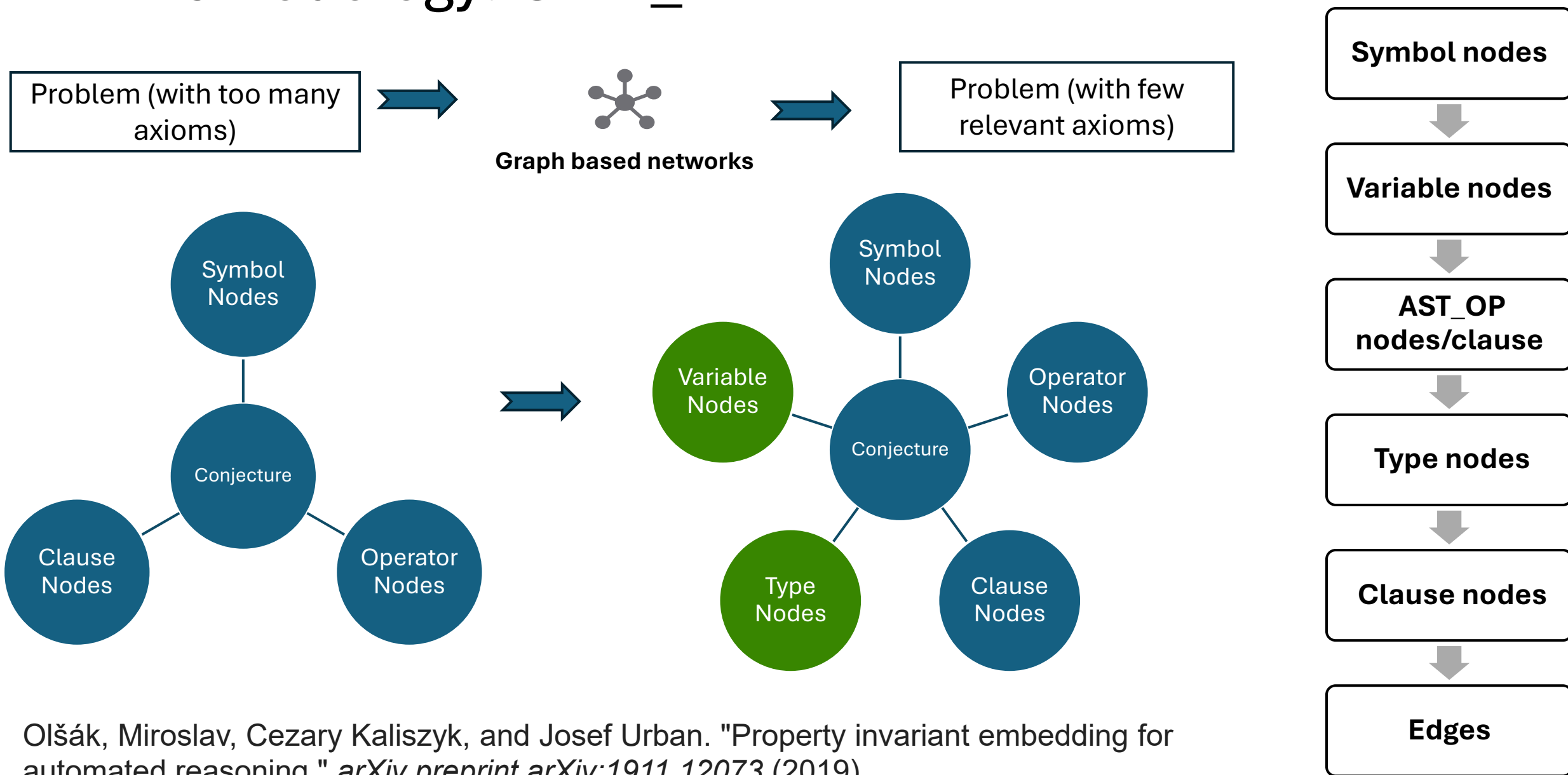
Graph-based networks



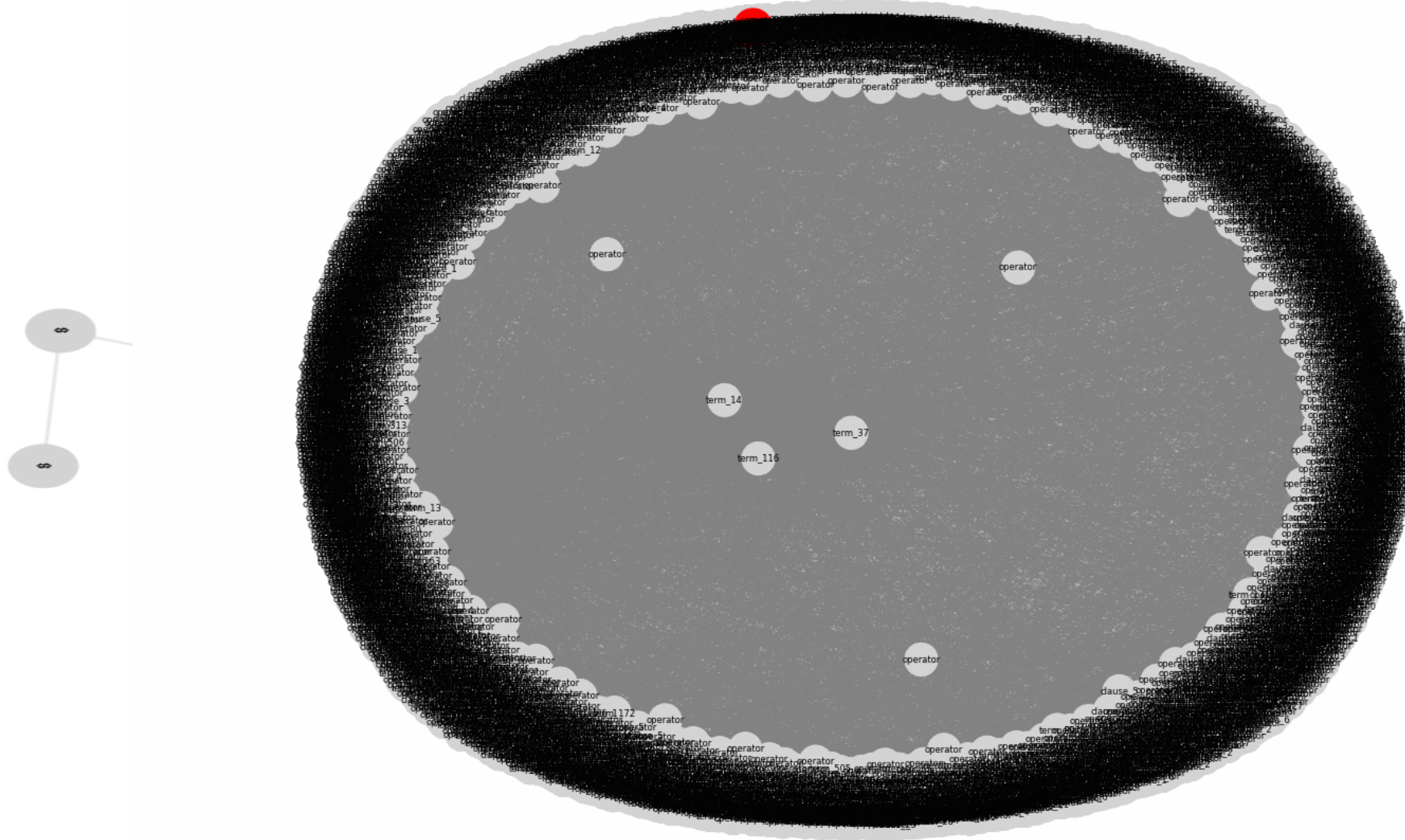
Sequence-based networks



Methodology: GNN_v1



GNN_V1



**Symbol
nodes**

**Variable
nodes**

**AST_OP
nodes/clause**

Type nodes

**Clause
nodes**

Edges

GNN_v1



Training

Graph of the training dataset

Fixed Trainig Graph

GNN Weights (random)

Training Embeddings

Loss & Gradients

Testing

New Test Graph for the test set....

Fixed Graph Structure

Frozen weights

Inference Embeddings

Predictions

Loss Function

1. Binary Cross Entropy Loss (BCE)
2. Focal Loss

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{Accuracy} = \text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{F-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Design

- Too many edges and too many nodes

Parameter Optimization

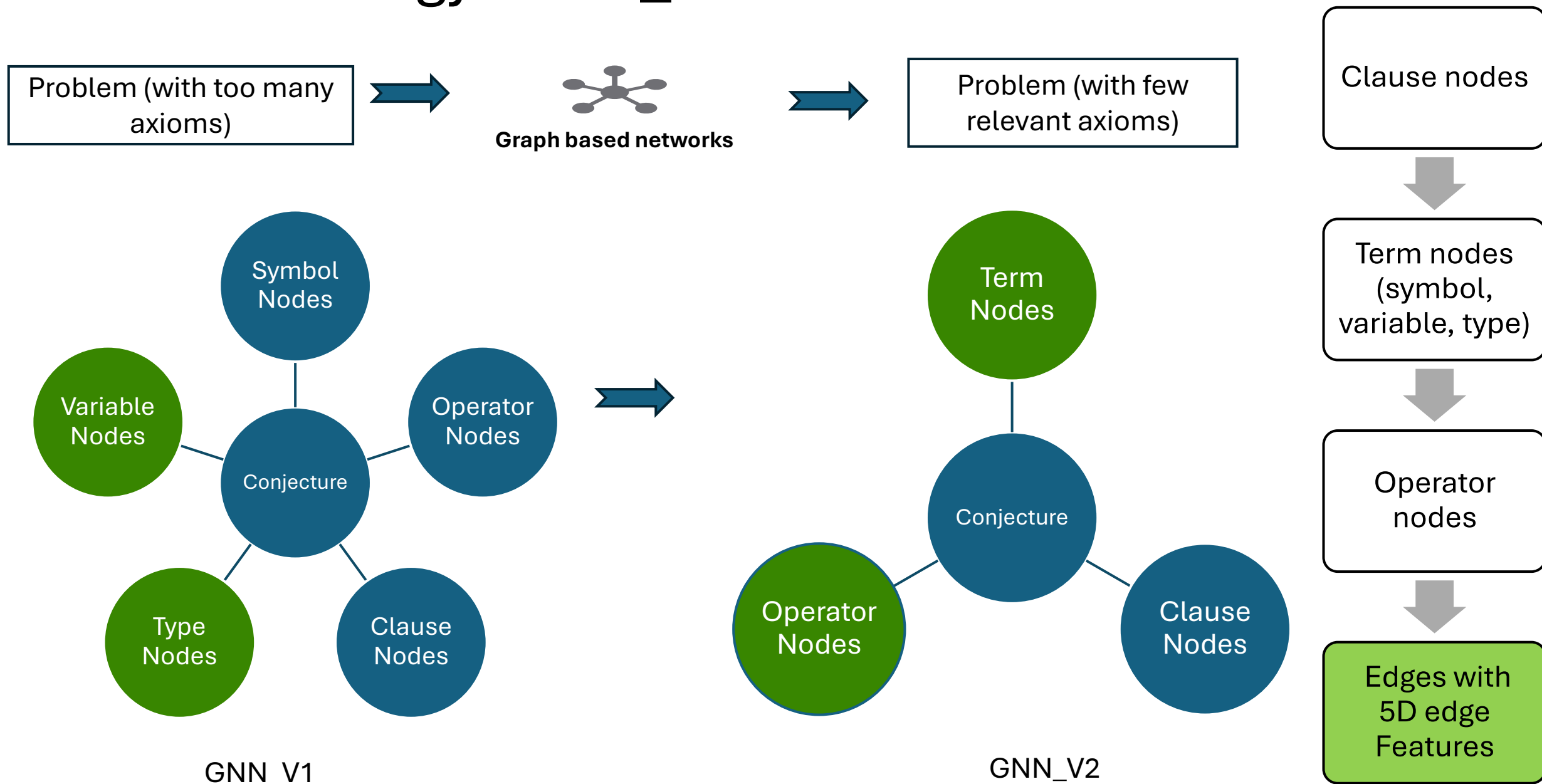
- Number of layers, Embedding size etc

Architecture details

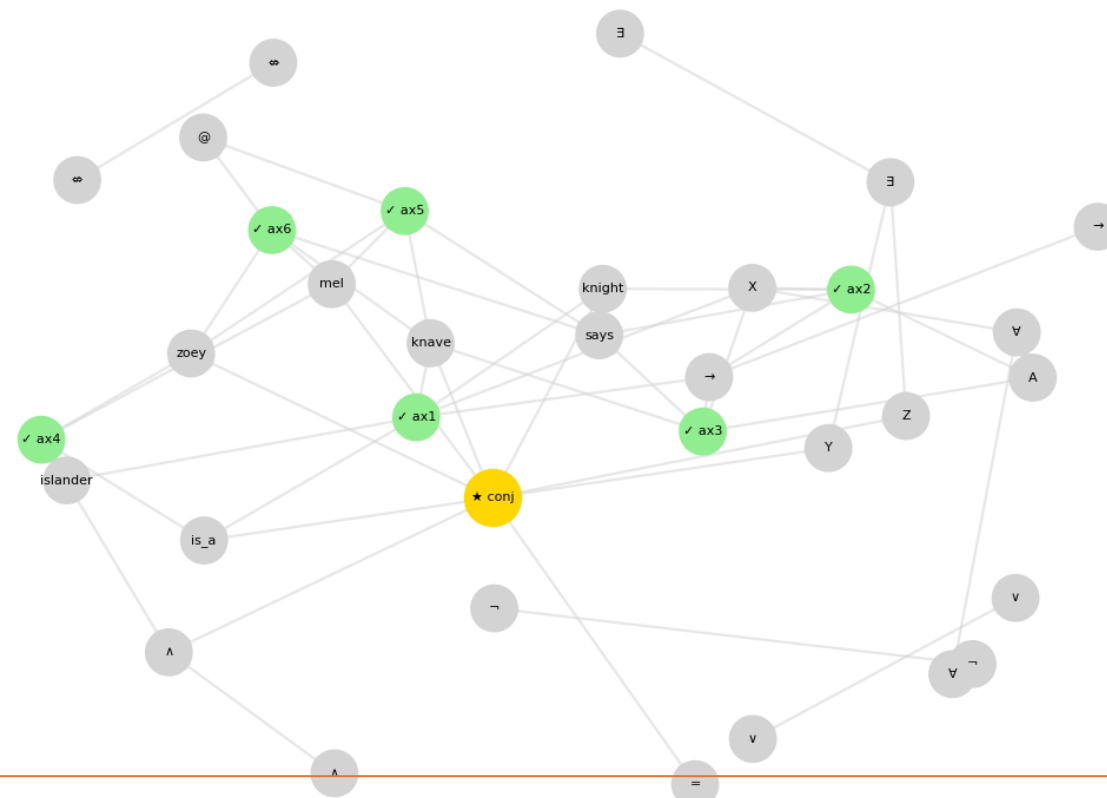
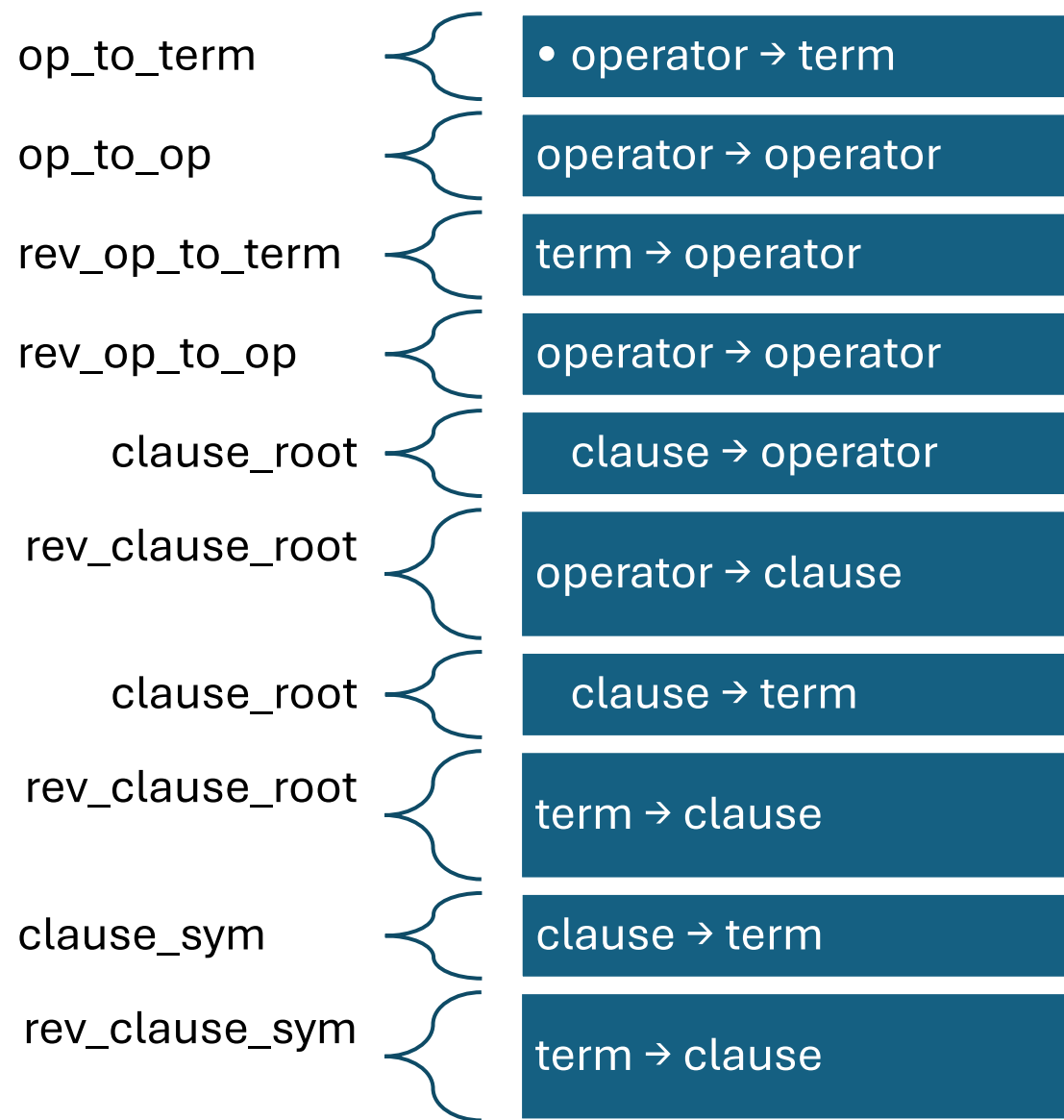
- Pre and postprocessing steps

Lin, Tsung-Yi, et al. "Focal loss for dense object detection." *Proceedings of the IEEE international conference on computer vision*. 2017.

Methodology: GNN_v2

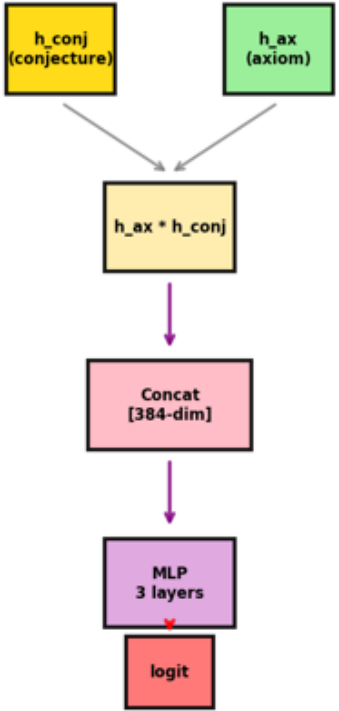
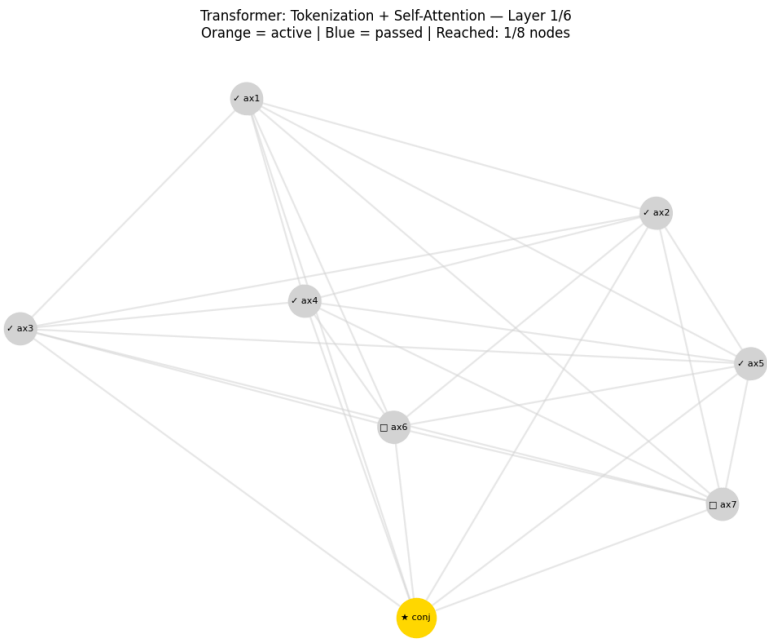
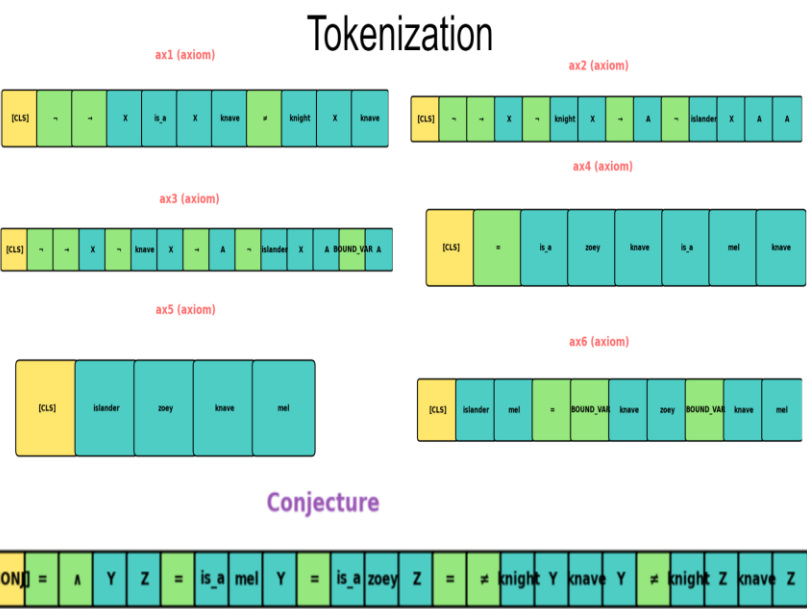
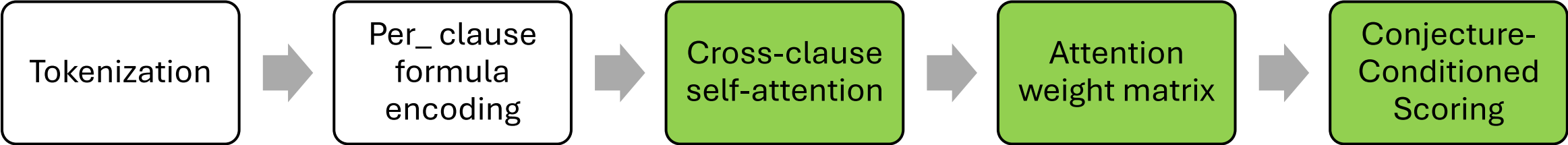


GNN_v2



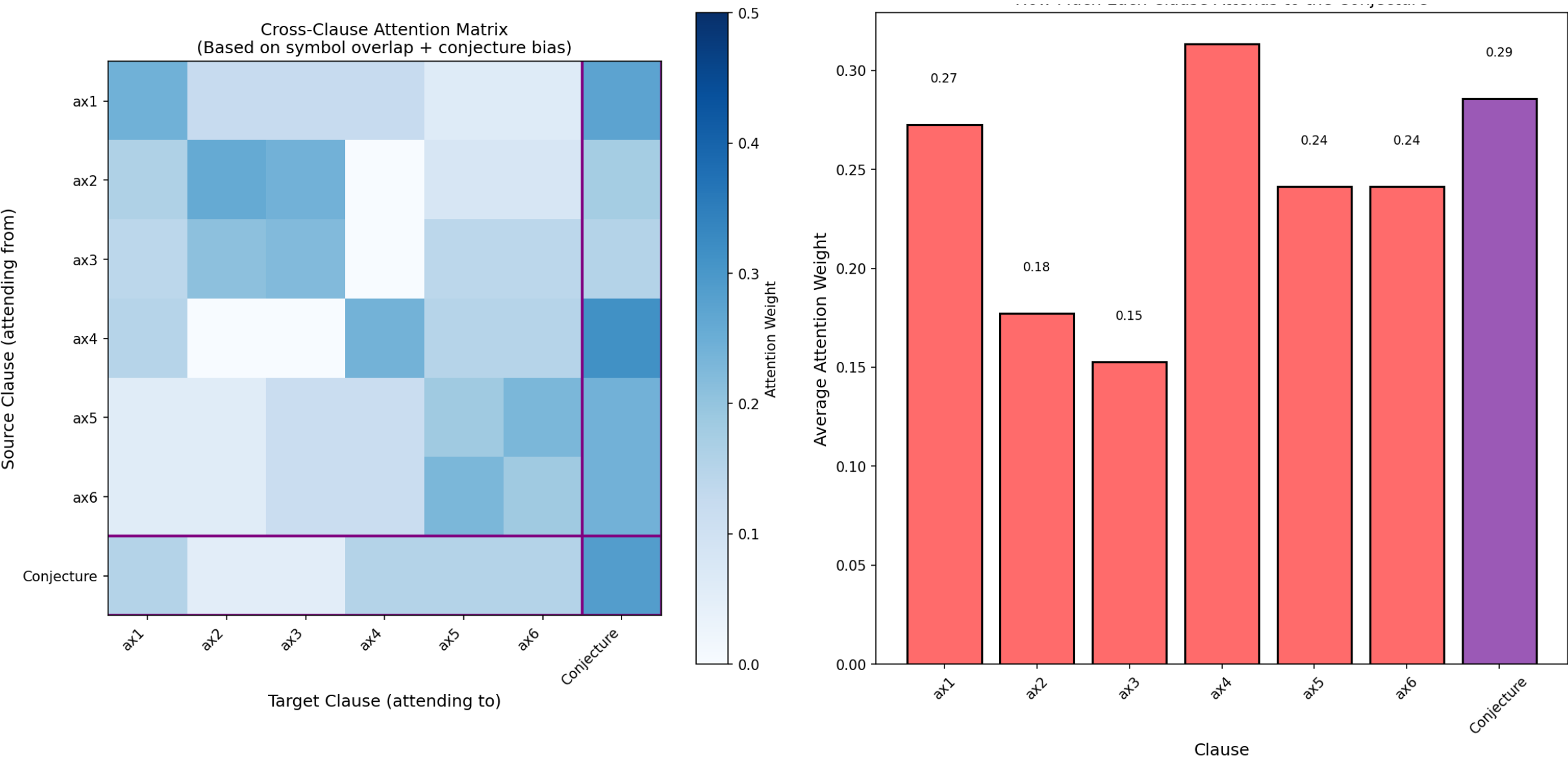
Design	Parameter Optimization	Architecture details
try a totally new design	Number of layers, Embedding size etc	Pre and postprocessing steps

Transformer

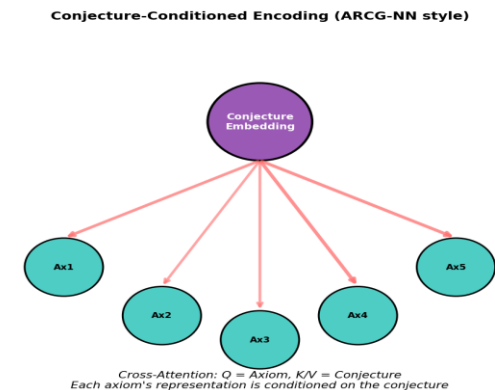
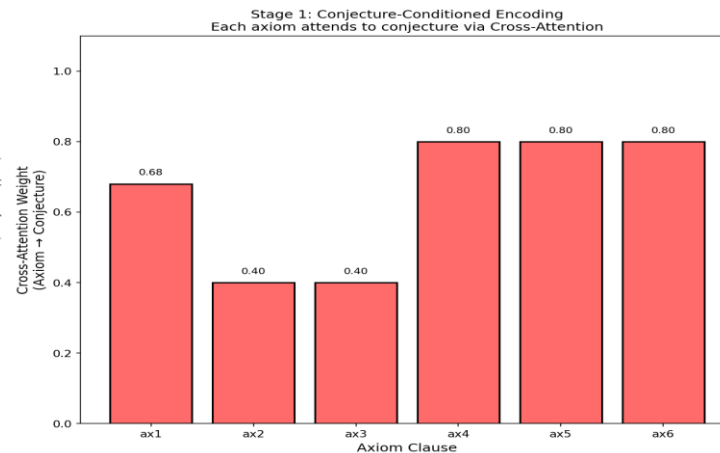
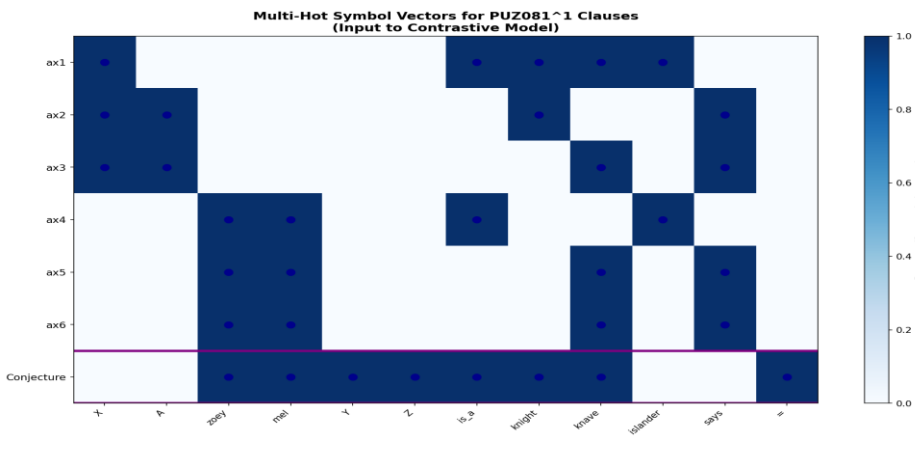
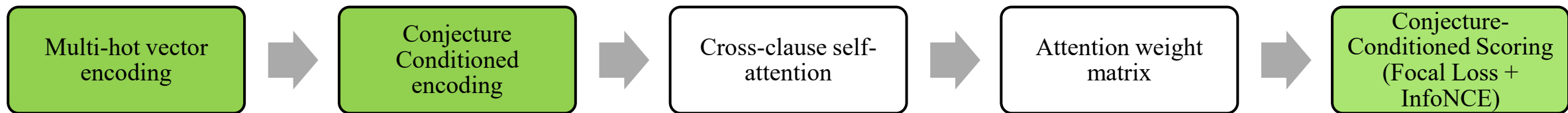
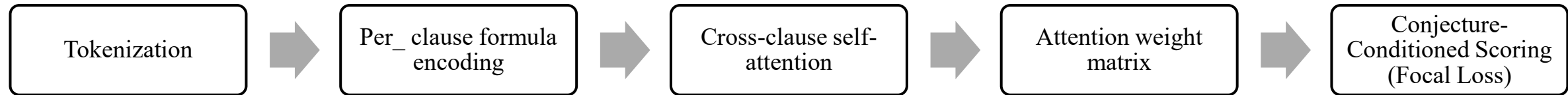


logit > 0 -> relevant | logit < 0 -> irrelevant

Transformer: Self-Attention Weight Matrix



Contrastive Transformer



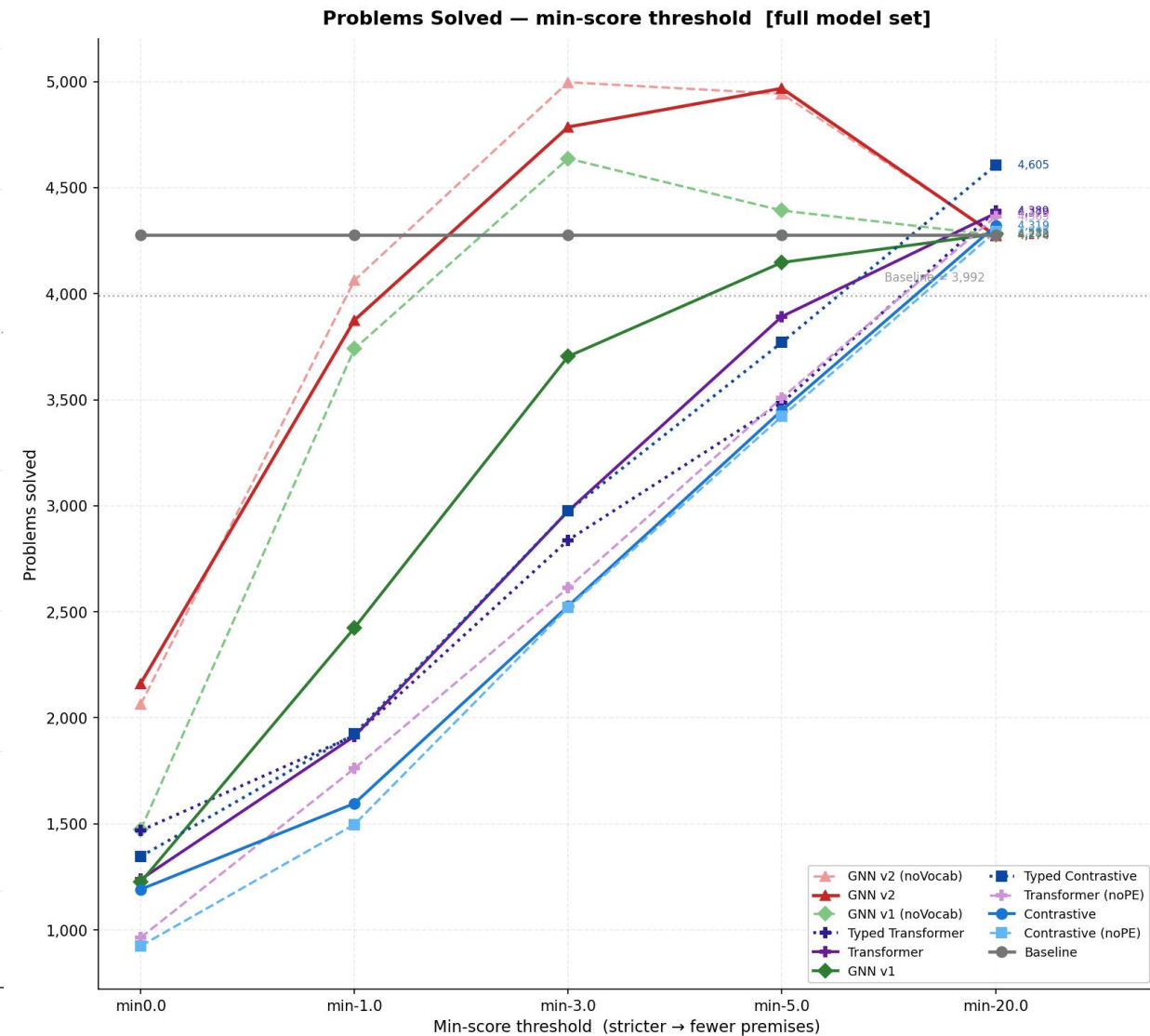
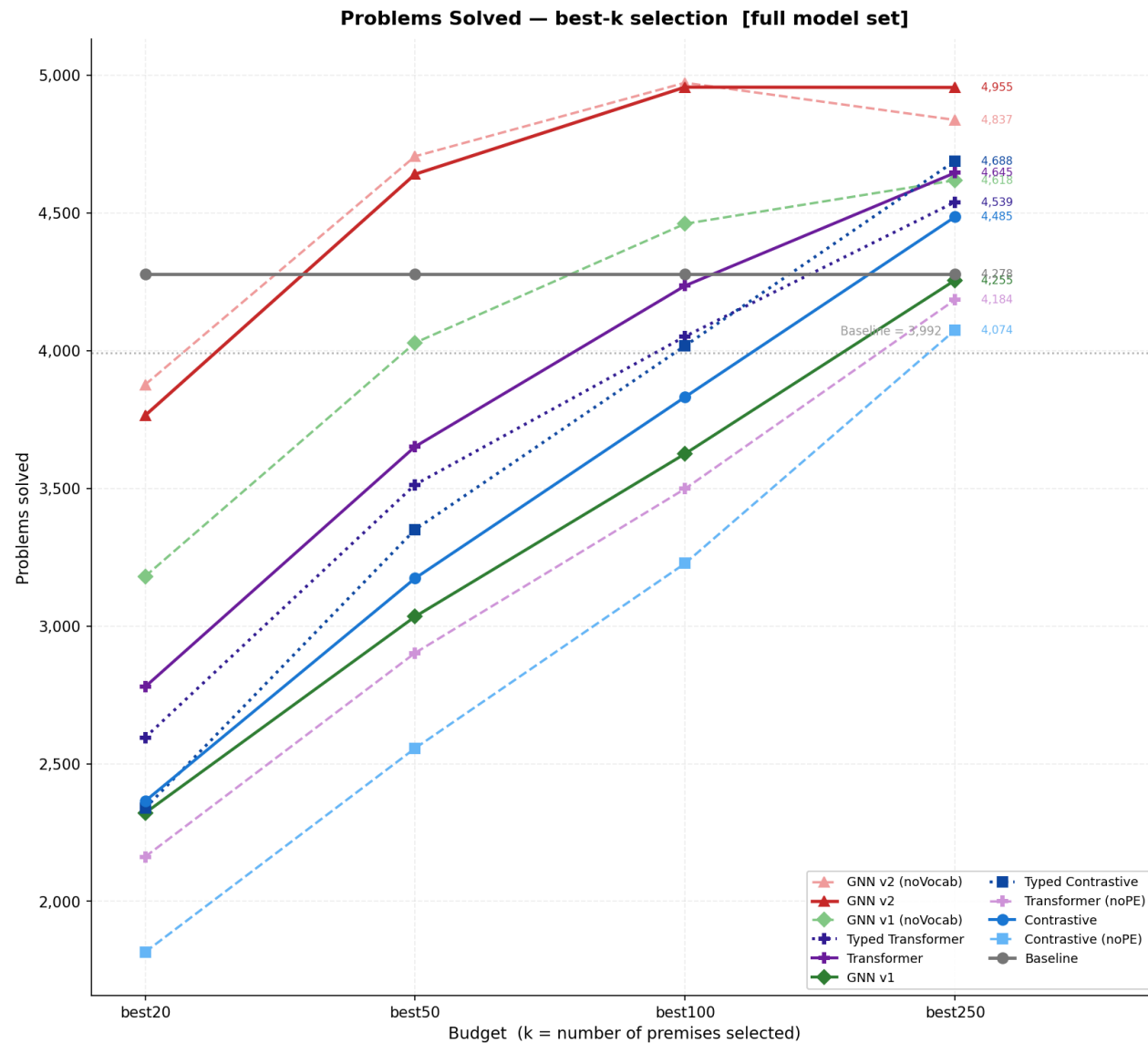
Preliminary - Experiments

- Dataset:
 - Training: 26,000 problems from Isabelle (80/20 train/val split)
 - Testing : 8900 holdout problems
- Two strategies:
 - Best X axioms (best 10,20)
 - Logit Score

Model

- Baseline (MePo): (E-prover, new-bool-7 strategy)
- GNN_V1
- GNN-V2
- Transformer
- Contrastive Transformer

Initial Results (T=10 s)



Experiment- On a bigger Data

- Dataset:
 - 118,000 problems from Isabelle
 - ~59,000 problems have proof
 - (90/10) – Train/Val split
 - 2000 Test problems
 - 5000 holdout problems
- **Strategies:**
 - Best X axioms (best 16,32, 64,128, 256, 512(baseline))
 - Normalized min logit scores(0.001, 0.01, 0.1, 0.25, 0.5, 0.75)
 - T- Time (10s)

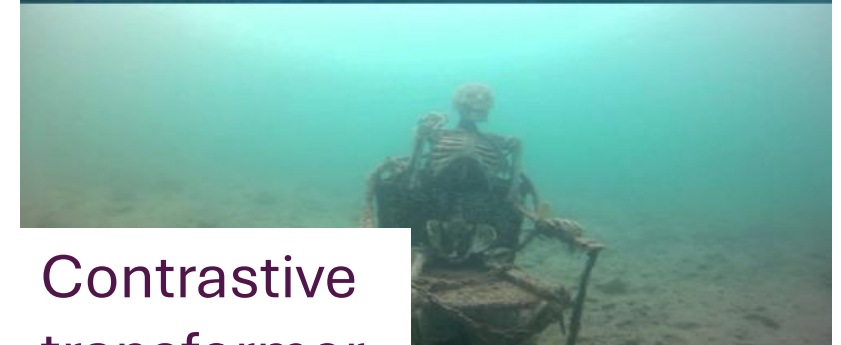


GNN_V2

Transformer

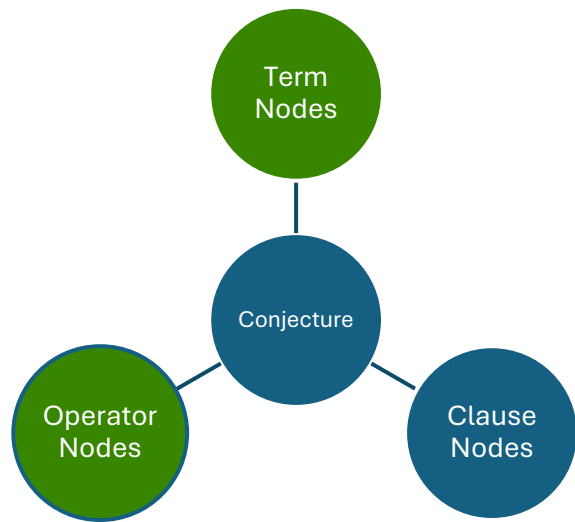


GNN_V1



Contrastive
transformer

**Nodes [symbol_id,
subtype, type_id]**



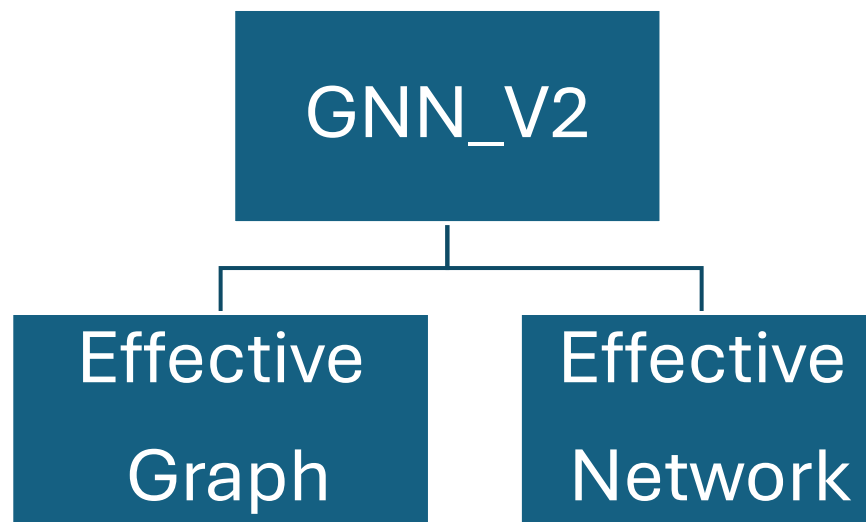
Edges

operator $\rightarrow$ term (rev)
operator $\rightarrow$ operator (rev)
term $\rightarrow$ operator
clause $\rightarrow$ operator (rev)
clause $\rightarrow$ term (rev)

Edge vectors

- **Role**
- **Direction**
- **Polarity**
- **Depth**
- **Shared or not**

- ✓ **Type information**
- ✓ **Lambda abstractions**
- ✓ **Partial applications**
- ✓ **Bound Variables**



- ✓ **Deeper**
- ✓ **Loss function**
- ✓ **Metrics**
- ✓ **Priors**

Example Problem: MESSAGE theory

```
137 lemma keysFor_empty [simp]: "keysFor {} = {}"  
138 by (unfold keysFor_def, blast)
```

```
( keysFor  
= ( ^ [X4: set_msg] :  
  ( image_nat_nat @ invKey  
    @ ( collect_nat  
      @ ^ [X5: nat] :  
        ? [X6: msg] : ( member_msg @ ( crypt @ X5 @ X6 ) @ X4 ) ) ) ) )
```

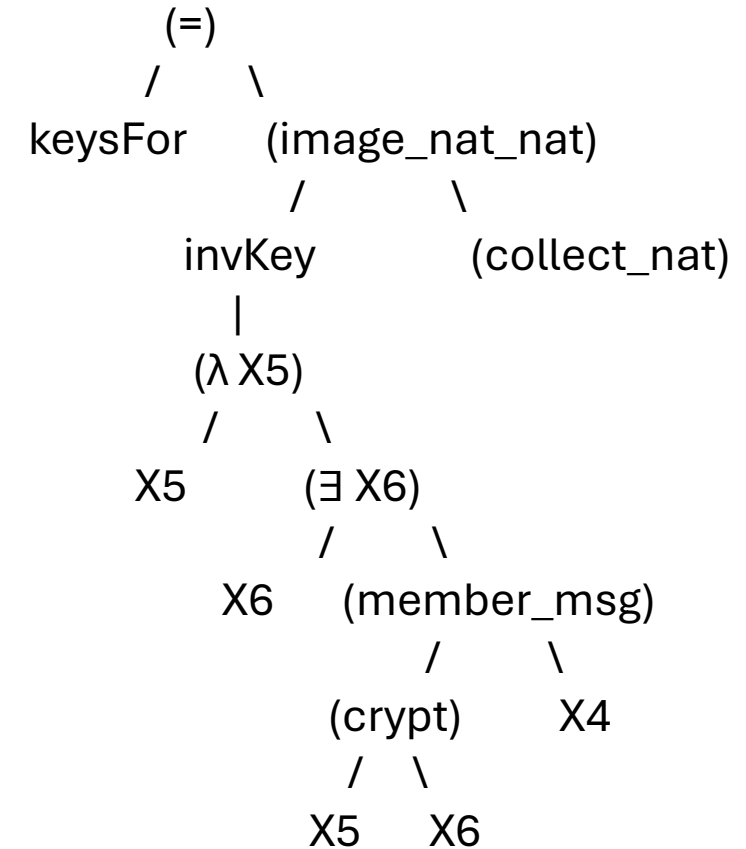
type declarations	169
axioms	635
conjecture	1
Axioms_needed	6

```
thf(conj_0, conjecture,  
  (((keysFor @ bot_bot_set_msg) = bot_bot_set_nat)))
```

- fact_3_keysFor__def — definition of keysFor
- fact_180_image__empty — image f {} = {} (nat instance)
- fact_2_empty__iff — nothing is a member of {} (nat)
- fact_96_bot__set__def — {} = Collect bot
- fact_99_bot__empty__eq — bot = ($\lambda x. x \in \{\}$)
- fact_1_empty__iff — nothing is a member of {} (msg)

Example Problem: fact_3_keysFor__def — definition of keysFor

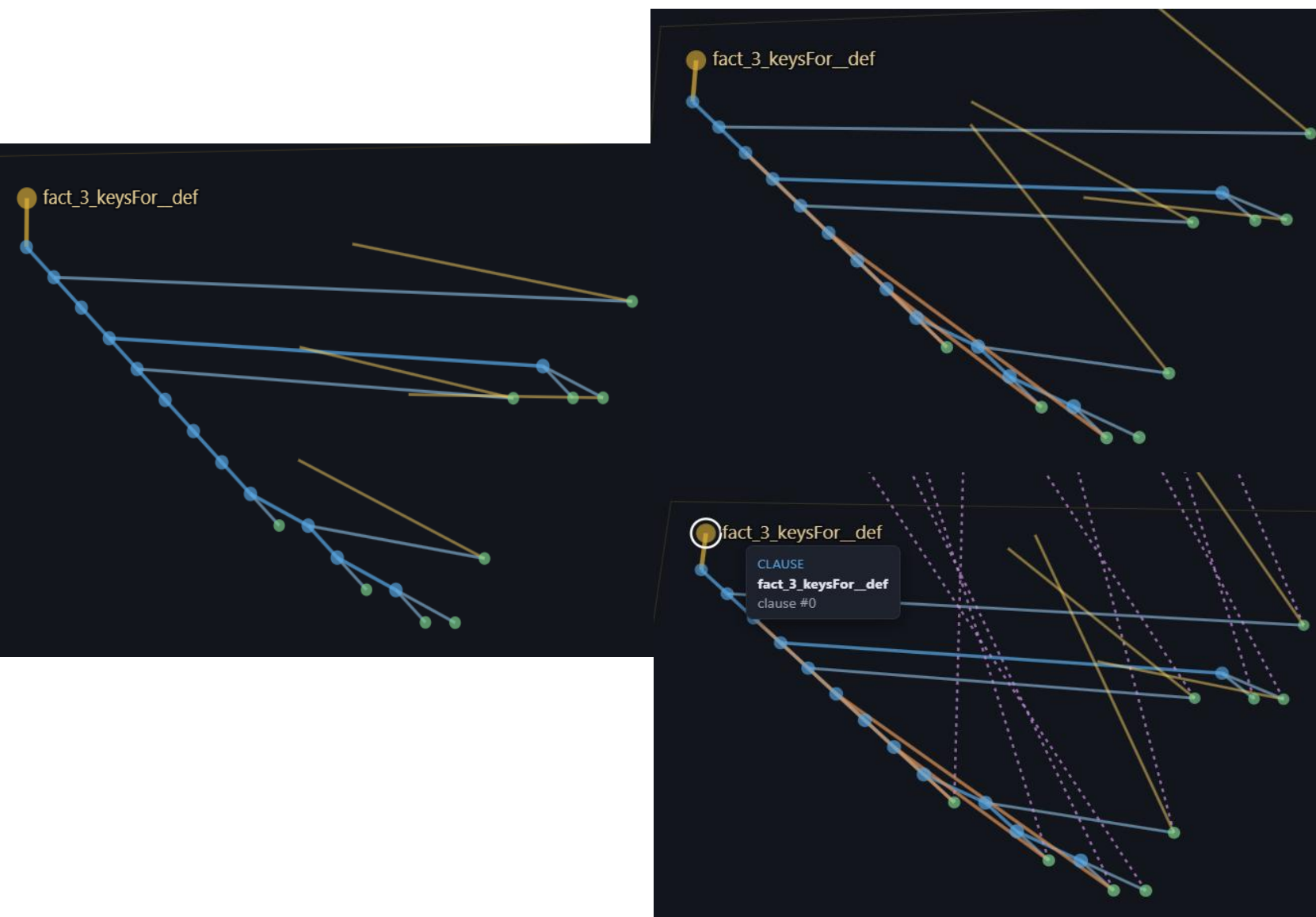
```
( keysFor
= ( ^ [X4: set_msg] :
  ( image_nat_nat @ invKey
    @ ( collect_nat
      @ ^ [X5: nat] :
        ? [X6: msg] : ( member_msg @ ( crypt @ X5 @ X6 ) @ X4 ) ) ) ) )
```



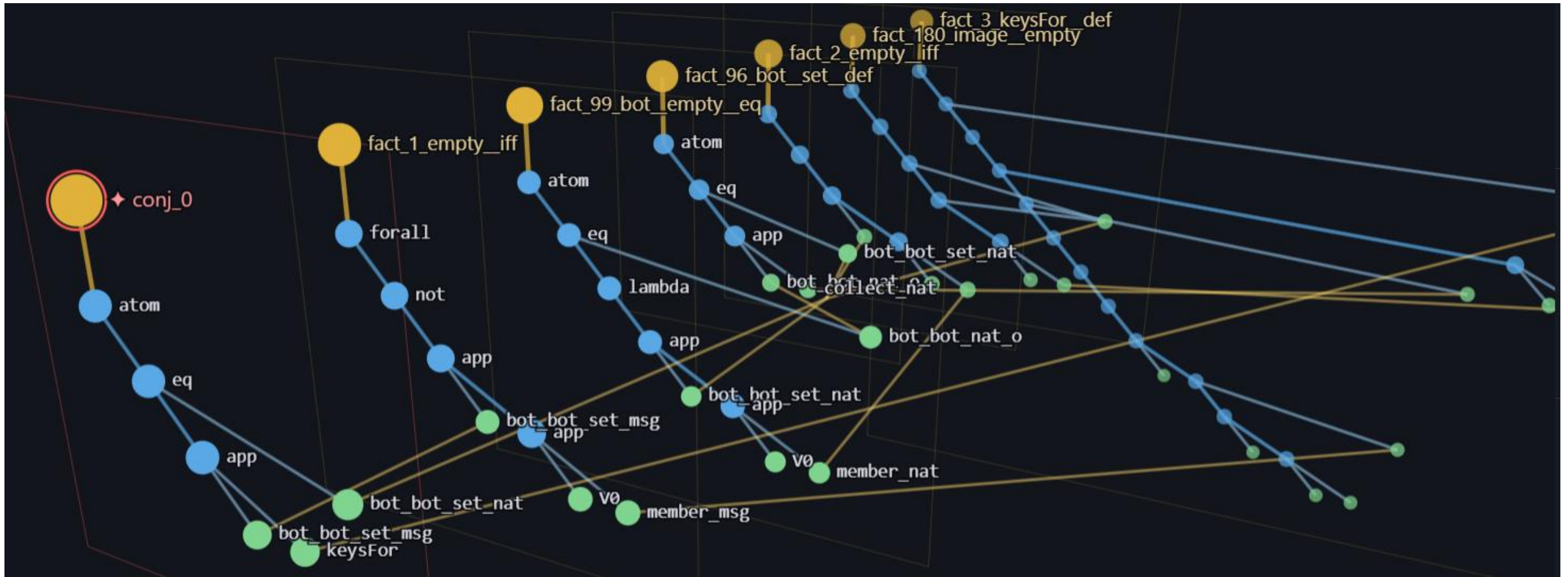
fact_3_keysFor__def

The control flow graph for 'fact_3_keysFor__def' is shown on a dark background. It starts with a yellow node at the top left, which branches into a blue node. This blue node then branches into a series of blue nodes that follow a diagonal path downwards and to the right. From the final blue node in this sequence, the graph branches into several green nodes, which are connected by lines, indicating the flow of execution through the function's logic.

Example Problem: fact_3_keysFor__def — definition of keysFor

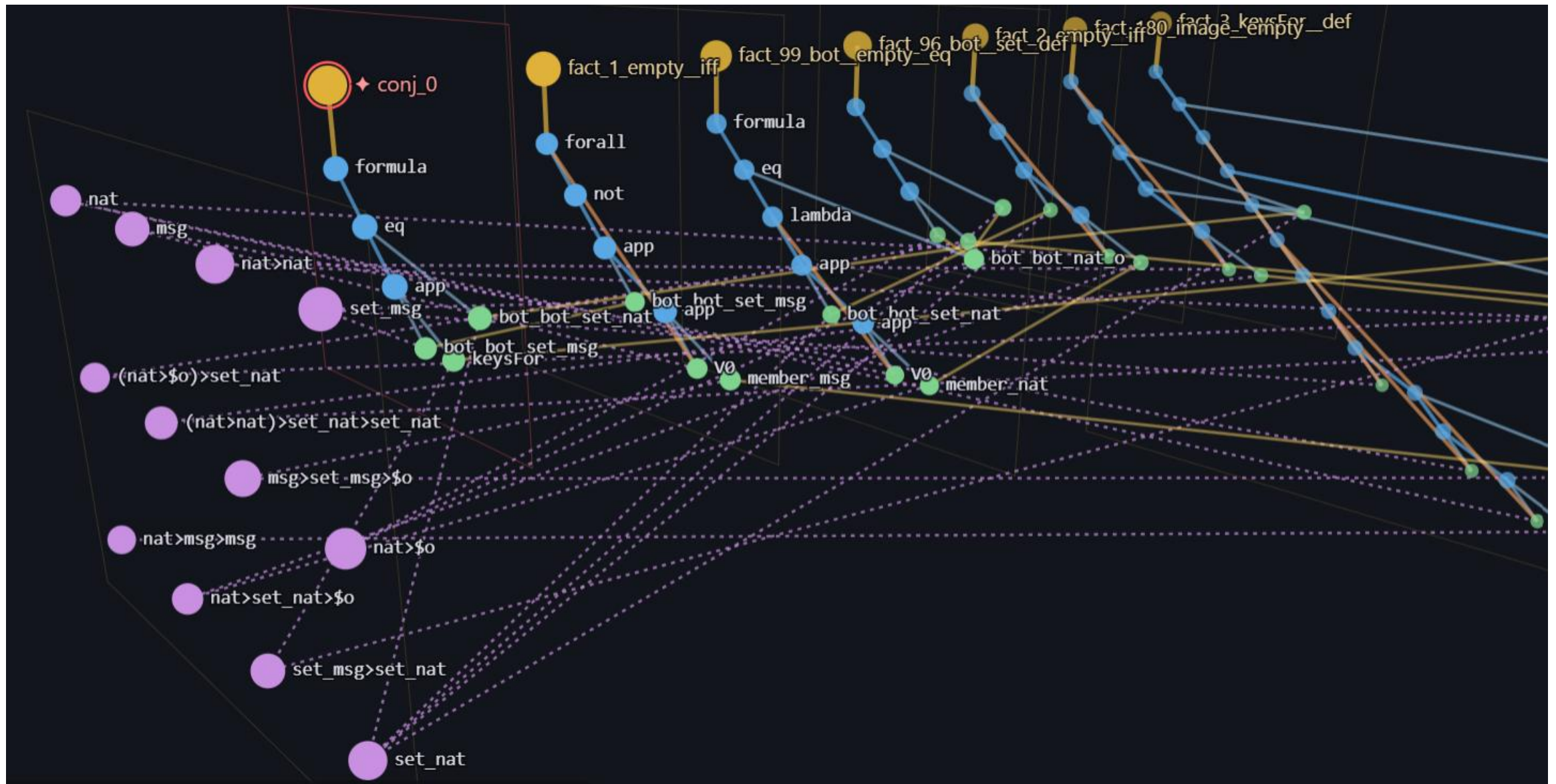


```
(=)
 /   \
keysFor (image_nat_nat)
 /      \
invKey   (collect_nat)
 |
(λ X5)
 /      \
X5      (∃ X6)
 /      \
X6      (member_msg)
 /      \
(crypt)  X4
 /      \
X5      X6
```



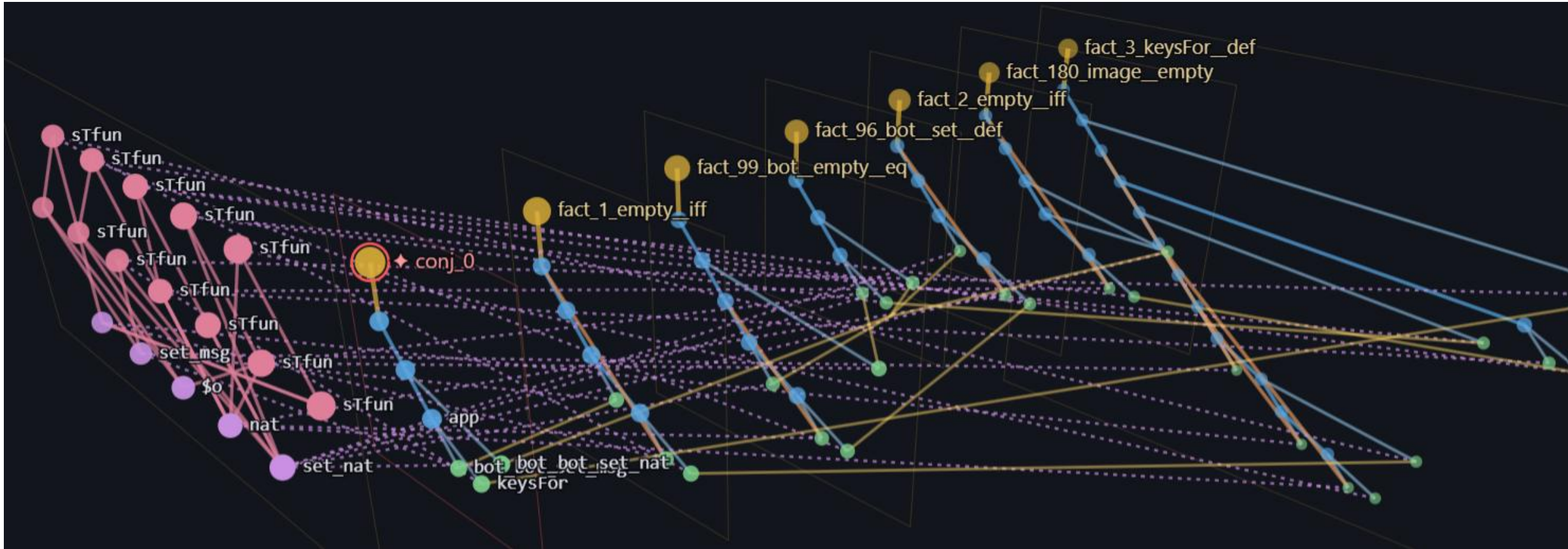
GNNV2

3 node sorts, 10 relations, 5-dim edge feature ([role, direction, polarity, depth and shared]



**GNNV2 +binding edges
+ type node**

One shared node per distinct resolved type string; has_type edges make “share a type” a first-class relation



**GNNV2 + binding edges
+ type node + type node
operator =
RICHGRAPH_GNN**

Compound function types get their own node, linked via `type_arg` — genuine structural sharing of type sub-components.

Now that we have the graph...

Effective Network

🏠 / `torch_geometric.nn` / `conv.GATv2Conv`

conv.GATv2Conv 🔗

```
class GATv2Conv ( in_channels: Union[int, Tuple[int, int]], out_channels: int, heads: int = 1, concat: bool = True, negative_slope: float = 0.2, dropout: float = 0.0, add_self_loops: bool = True, edge_dim: Optional[int] = None, fill_value: Union[float, Tensor, str] = 'mean', bias: bool = True, share_weights: bool = False, residual: bool = False, **kwargs ) [source]
```

Bases: `MessagePassing`

The GATv2 operator from the “How Attentive are Graph Attention Networks?” paper, which fixes the static attention problem of the standard `GATConv` layer. Since the linear layers in the standard GAT are applied right after each other, the ranking of attended nodes is unconditioned on the query node. In contrast, in `GATv2`, every node can attend to any other node.

$$\mathbf{x}'_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{i,j} \Theta_t \mathbf{x}_j,$$

where the attention coefficients $\alpha_{i,j}$ are computed as

$$\alpha_{i,j} = \frac{\exp(\mathbf{a}^\top \text{LeakyReLU}(\Theta_s \mathbf{x}_i + \Theta_t \mathbf{x}_j))}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(\mathbf{a}^\top \text{LeakyReLU}(\Theta_s \mathbf{x}_i + \Theta_t \mathbf{x}_k))}.$$

If the graph has multi-dimensional edge features $\mathbf{e}_{i,j}$, the attention coefficients $\alpha_{i,j}$ are computed as

$$\alpha_{i,j} = \frac{\exp(\mathbf{a}^\top \text{LeakyReLU}(\Theta_s \mathbf{x}_i + \Theta_t \mathbf{x}_j + \Theta_e \mathbf{e}_{i,j}))}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(\mathbf{a}^\top \text{LeakyReLU}(\Theta_s \mathbf{x}_i + \Theta_t \mathbf{x}_k + \Theta_e \mathbf{e}_{i,k}))}.$$

1. Algebraic-property prior

category	comm	assoc	idem	distrib	invert	quant	conn
app	0	0	0	0	0	0	0
$\forall/\exists$	0	0	0	0	0	1	0
λ	0	0	0	0	0	0	0
$\supset$	0	0	0	0	0	0	1
$\wedge/\vee$	1	1	1	1	0	0	1
$\neg$	0	0	1	0	0	0	0
$\leftrightarrow/\oplus$	1	1	0	0	0	0	1
eq	1	0	0	0	0	0	0

2. Relational message passing
3. Gated Node Updates
4. Global-pooling per node sort
5. Cross-Clause attention
6. Deep supervision

GNNV2 + network upgrades = RICH_NETWORK_GNN

Loss Function and Evaluation

Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

F-score = $2 \times (Precision \times Recall) / (Precision + Recall)$

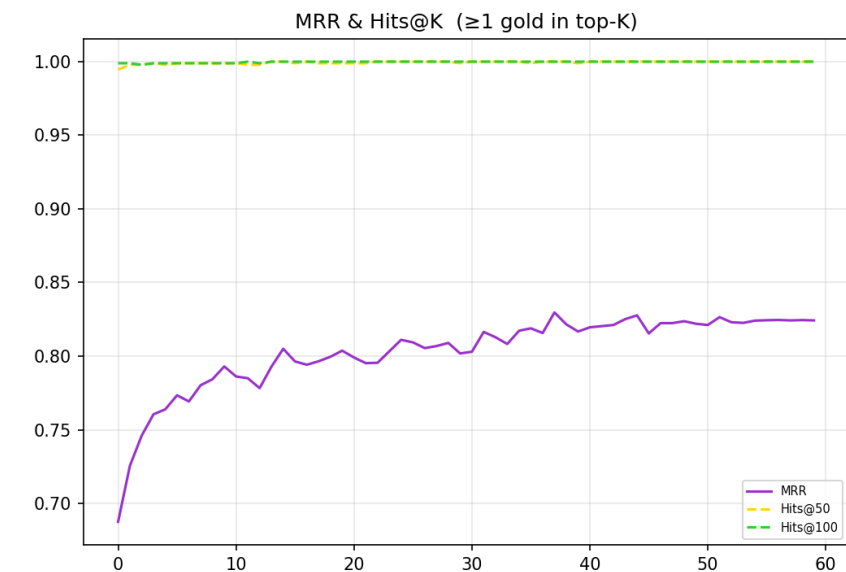
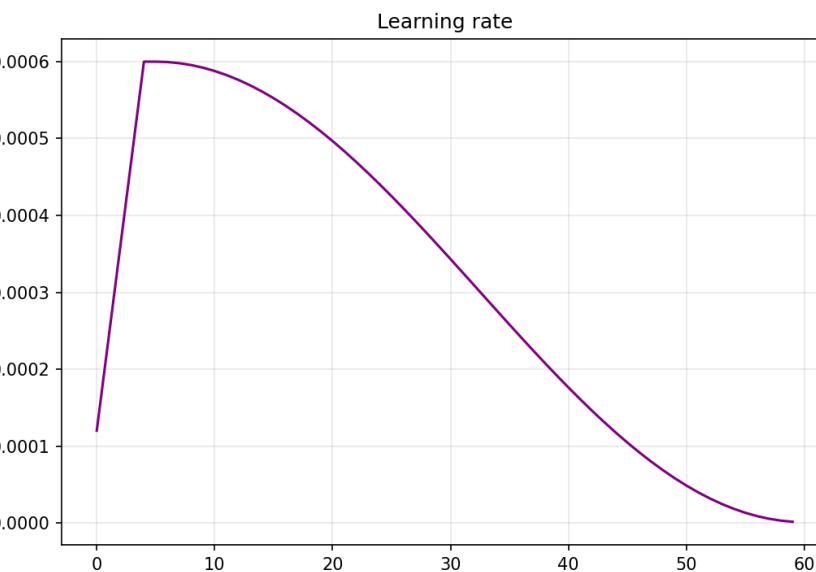
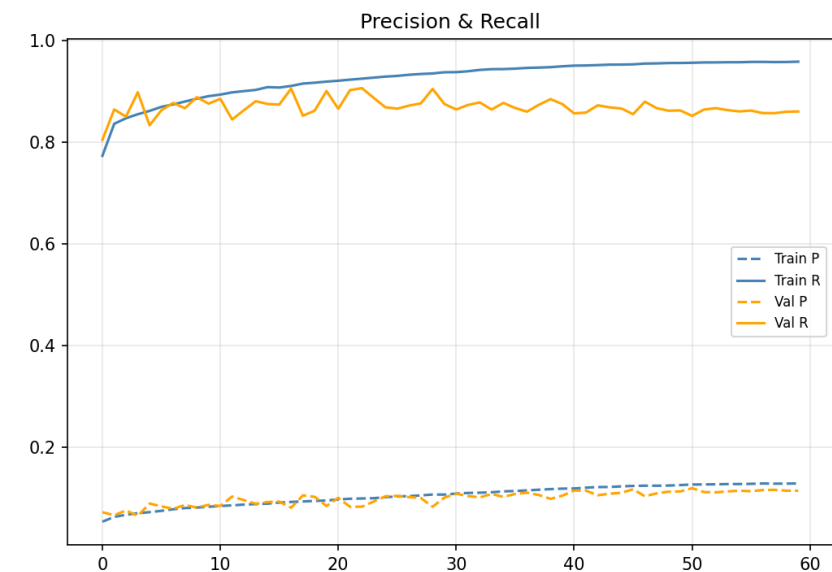
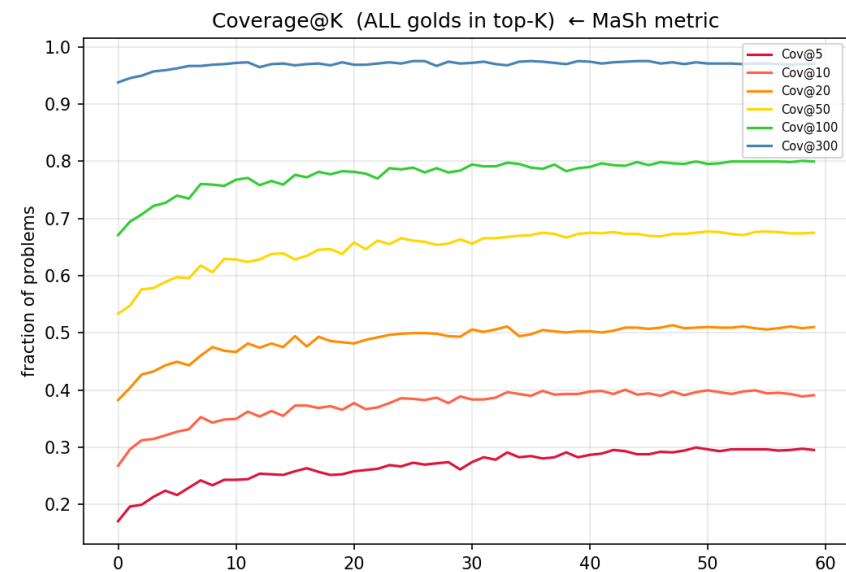
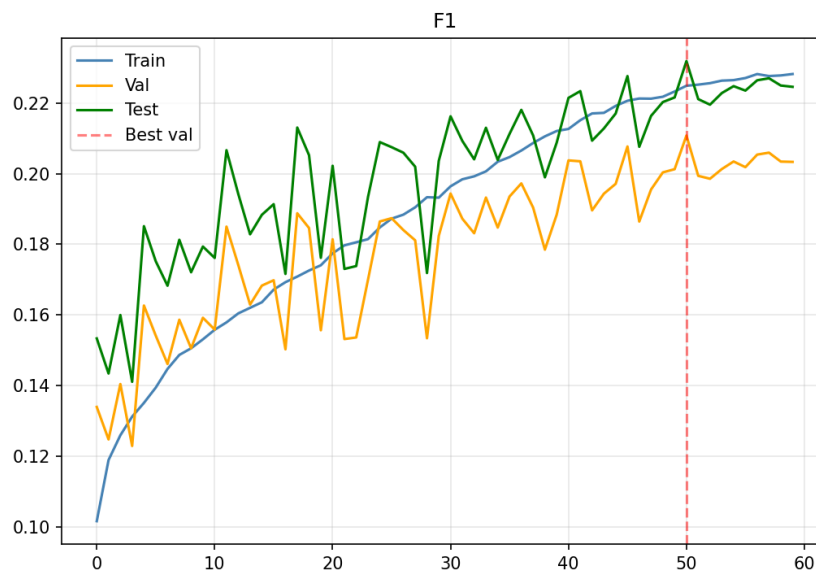
$$\text{Mean rank} = \frac{1}{\sum_p |G_p|} \sum_p \sum_{g \in G_p} \text{rank}(g)$$

$$\text{MRR} = \frac{1}{|P|} \sum_p \frac{1}{\text{rank}_{\min}(p)}$$

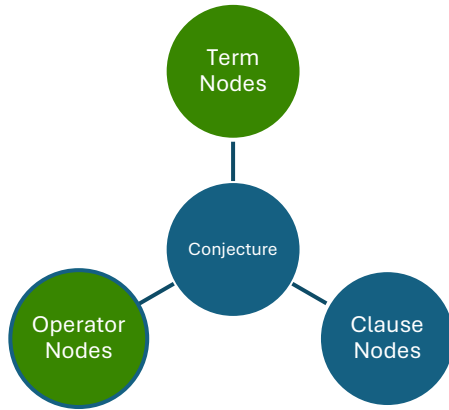
$$\text{Hits@K} = \frac{1}{|P|} \sum_p \mathbb{I}[\text{rank}_{\min}(p) \leq K]$$

$$\text{Coverage@K} = \frac{1}{|P|} \sum_p \mathbb{I}[\text{rank}_{\max}(p) \leq K]$$

- **Mean rank** : measures the average position of relevant axioms in the ranked list; lower values indicate the model consistently ranks useful premises near the top.
- **Mean Reciprocal Rank (MRR)** measures how high the first relevant axiom appears, averaged across problems. MRR favors models that put at least one useful premise very early.
- **Hits@K** measures whether the highest-ranked relevant axiom appears within the top K. This captures whether the model can provide the single most important premise.
- **Coverage@K** measures whether all relevant axioms for a problem appear within the top K



Nodes



Edges

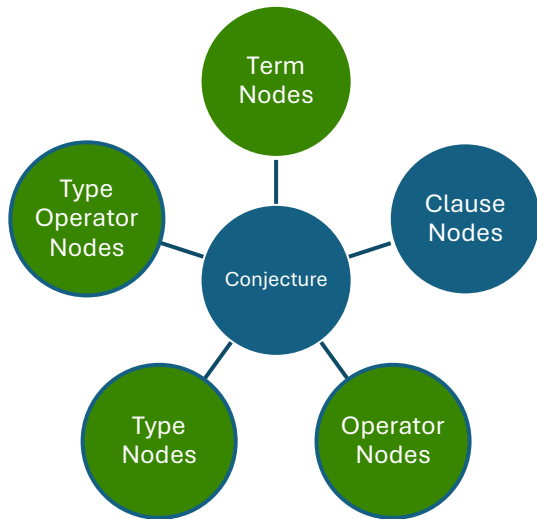
$\text{operator} \rightarrow \text{term (rev)}$
 $\text{operator} \rightarrow \text{operator (rev)}$
 $\text{term} \rightarrow \text{operator}$
 $\text{clause} \rightarrow \text{operator (rev)}$
 $\text{clause} \rightarrow \text{term (rev)}$

Edge features

- Role
- Direction
- Polarity
- Depth
- Shared or not

GNNV2

Nodes



$+ (\text{bound}) \text{operator} \rightarrow \text{var (rev)}$
 $+ (\text{has type}) \text{term} \rightarrow \text{type (rev)}$
 $+ (\text{has type}) \text{term} \rightarrow \text{type_op (rev)}$
 $+ (\text{type_arg}) \text{type_op} \rightarrow \text{type (rev)}$
 $+ (\text{type_arg}) \text{type_op} \rightarrow \text{type_op (rev)}$

+ Pos encoding

$P \rightarrow Q$

$\text{pos}(P) = (0.0, 1.0, 0.0)$

$\text{pos}(Q) = (1.0, 0.0, 1.0)$

RICH_GRAPH_GNN

Back to the Experiment

- Dataset:
 - 118,000 problems from Isabelle
 - ~59,000 problems have proof
 - (90/10) – Train/Val split
 - 2000 Test problems
 - 5000 holdout problems
- Strategies:
 - Best X axioms
 - Normalized min logit scores
 - T- Time (10s)



RICHGRAPH_GNN

RICH_NETWORK_GNN



Transformer,
S-transformer,
Hybrid_GNN etc



GNN_V1,
Contrastive
transformer

Results

47.7% :

Baseline new_bool_7
(2,382 / 4,994)

61.3% :

NETWORK_RICH_GNN
total cover : strategies
(3,062 / 4,994)

55%

GRAPH_RICH_GNN
best single deployable
strategy, min0.25 (2,747)

Model	Best strategy	Best score	Total cover
Transformer	best256	2,410	2,847
Gnn_Sine_hybrid	best256	2,552	2,876
GNN_V2	best250	2,589	2,881
GNN_V2+ Binding_edges	best128	2,619	2,974
GRAPH_RICH_GNN	min0.25	2,747	3,056
NETWORK_RICH_GNN	min0.25	2,743	3,062
GRAPH_&_NETWORK_RICH_GNN	min0.25	2,725	3,043

Ablation Study

55.5% GRAPH_RICH_GNN_NoType
best single deployable strategy, min0.25 (2,775)

Configuration	Component removed	Total cover	Best strategy	Best score
No clausesym	clause_sym shortcut edges	2,932	min0.25	2,642
No attention	GATv2 attention ($\rightarrow$ SAGEConv)	3,045	min0.2	2,696
No binding edges	binds / rev_binds edges	3,028	min0.25	2,743
No pos	positional edge features	3,046	min0.25	2,745
No type, No binding_edges, Noattn	type+binds AND attention	3,056	min0.3	2,746
Graph_Rich_GNN + logic_property	+ logic-property embed (addition)	3,070	min0.15	2,751
Notype_nobinding edges	type + binds (combined)	3,073	min0.3	2,765
No_notype	type / type_op nodes+edges	3,033	min0.25	2,775
Graph_Rich_GNN	— (reference, tuned)	3,056	min0.25	2,747

Conclusion

2382 : problems solved by baseline

+ **365** : solved by **GRAPH_RICH_GNN**, best single deployable strategy, min0.25 (2,747)

+ **680**: solved by **NETWORK_RICH_GNN**, total cover : strategies (3,062 / 4,994)

+ **779**: Greedy cover results

GRAPH_RICH_GNN	min0.25	2,747	3,056
NETWORK_RICH_GNN	min0.25	2,743	3,062
GRAPH_&_NETWORK_RICH_GNN	min0.25	2,725	3,043

Future work

- Is binding information useful at all?
- How come no attention affected it negatively where there was type info...
- How important is Type information in HOL?

